

ANÁLISIS DEL RETO

Juan Camilo Cancelado, 202410123, j.canceladod@unaindes.edu.co

Gabriela Gomez, 202420506, g.gomezh2@uniandes.edu.co

Pedro Archila, 202421572, p.archila@unaindes.edu.co

Requerimiento Ejemplo

Descripción

```
def get_data(data_structs, id):  
    """  
    Retorna un dato a partir de su ID  
    """  
    pos_data = lt.isPresent(data_structs["data"], id)  
    if pos_data > 0:  
        data = lt.getElement(data_structs["data"], pos_data)  
        return data  
    return None
```

Este requerimiento se encarga de retornar un dato de una lista dado su ID. Lo primero que hace es verificar si el elemento existe. Dado el caso que exista, retorna su posición, lo busca en la lista y lo retorna. De lo contrario, retorna None.

Entrada	Estructuras de datos del modelo, ID.
Salidas	El elemento con el ID dado, si no existe se retorna None
Implementado (Sí/No)	Si. Implementado por Juan Andrés Ariza

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Buscar si el elemento existe (isPresent)	$O(n)$
Obtener el elemento (getElement)	$O(1)$
TOTAL	$O(n)$

Pruebas Realizadas

Las pruebas realizadas fueron realizadas en una maquina con las siguientes especificaciones. Los datos de entrada fueron el ID 1.

Procesadores	AMD Ryzen 7 4800HS with Radeon Graphics
Memoria RAM	8 GB
Sistema Operativo	Windows 10

Entrada	Tiempo (ms)
small	0.05
5 pct	0.33
10 pct	1.28
20 pct	2.54
30 pct	4.98
50 pct	7.51
80 pct	13.81
large	25.97

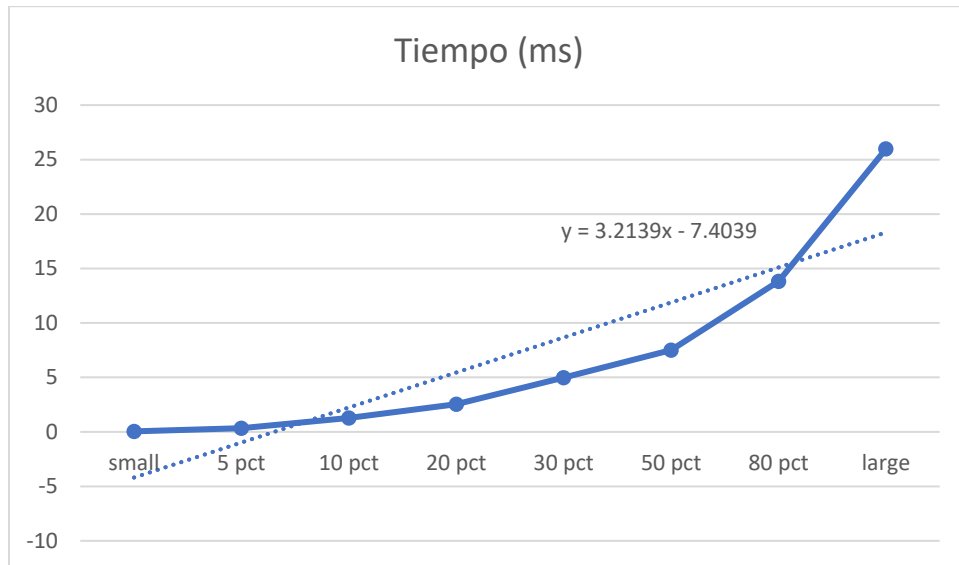
Tablas de datos

Las tablas con la recopilación de datos de las pruebas.

Muestra	Salida	Tiempo (ms)
small	Dato1	0.05
5 pct	Dato2	0.33
10 pct	Dato3	1.28
20 pct	Dato4	2.54
30 pct	Dato5	4.98
50 pct	Dato6	7.51
80 pct	Dato7	13.81
large	Dato8	25.97

Graficas

Las gráficas con la representación de las pruebas realizadas.



Análisis

A pesar de que obtener un elemento en un *ArrayList*, dada su posición, tiene complejidad constante, la implementación de este requerimiento tiene un orden lineal $O(n)$. Esto debido a que, lo primero que se hace es verificar si el elemento hace parte de la lista. Específicamente, a la hora de buscar un elemento en una lista, en el peor de los casos es necesario recorrer toda la lista, es decir, complejidad lineal.

Este comportamiento se puede evidenciar experimentalmente en la gráfica. Ya que, gracias a que los datos no se encuentran tan dispersos con respecto a la línea de tendencia, la curva coincide con el comportamiento lineal esperado.

Requerimiento <<1>>

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

Breve descripción de como abordaron la implementación del requerimiento

Entrada	El catálogo con la información del CVS, el año que quiere filtrar el usuario
Salidas	El tiempo de ejecución, el numero total de registros que entraban al filtro del año esperado y un array con la información (año de recopilación, fecha de carga, tipo de fuente, nombre del departamento, tipo de producto, unidad de medición y valor de medición) del último registro. Si no hay registros en ese año se retorna None.
Implementado (Sí/No)	Si - Grupal

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Saca el tamaño de la lista que tiene los años de recolección de datos	O(1)
Inicia una nueva lista	O(1)
Busca los elementos que tienen el mismo año que el dado por el usuario con get_element()	O(1) pero como está dentro de un for se convierte en O(N)
Añade al final de la lista creada anteriormente el dato que cumple con el requisito	O(1)
Usa get_element() para encontrar los elementos de la misma fila del elemento que cumple el requisito	O(1)
TOTAL	O(N)

Pruebas Realizadas

- Año buscado: 2010

Procesador	Apple M1
Memoria RAM	8 GB
Sistema Operativo	macOS Sequoia Versión 15.1

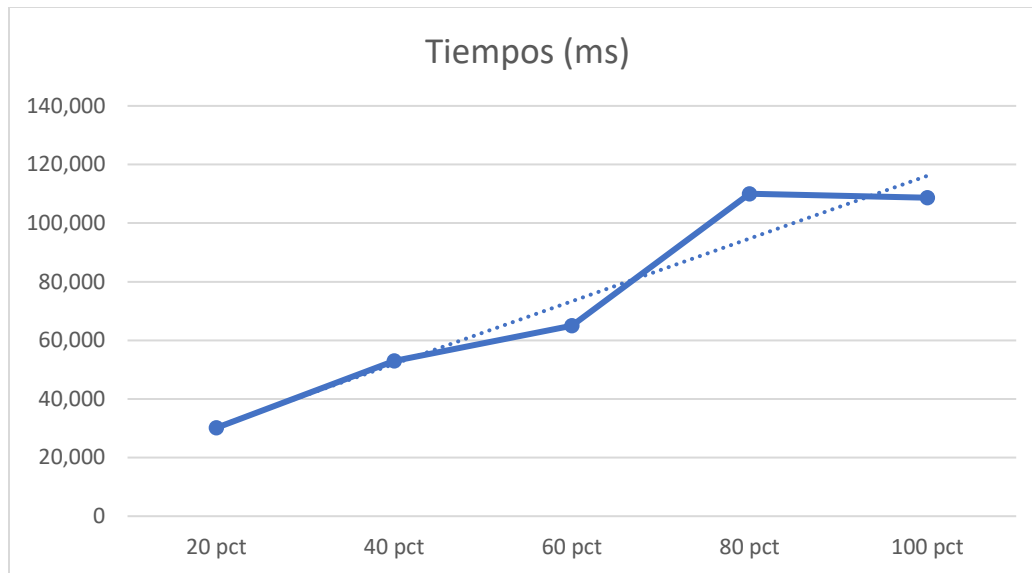
Tablas de datos

Las tablas con la recopilación de datos de las pruebas.

Entrada	Tiempo (ms)
20%	30.154 ms
40%	52.932 ms
60%	64.95 ms
80%	110.026 ms
100%	108.62 ms

Graficas

Las gráficas con la representación de las pruebas realizadas.



Análisis

Análisis de resultados de la implementación, tener cuenta las pruebas realizadas y el análisis de complejidad.

El análisis de complejidad indica que el algoritmo tiene un comportamiento **$O(N)$** , ya que recorre la lista completa para verificar si los registros cumplen con el año de filtrado. En la gráfica, se observa un crecimiento progresivo del tiempo de ejecución a medida que aumenta la cantidad de datos procesados. La línea de tendencia muestra una relación lineal, lo que confirma que el tiempo de ejecución es directamente proporcional al número de registros evaluados. No se observan grandes desviaciones en la gráfica, lo que sugiere que la implementación mantiene un comportamiento estable.

Requerimiento <<2>>

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

Breve descripción de como abordaron la implementación del requerimiento

Entrada	El catálogo con la información del CVS, el estado que quiere filtrar el usuario
Salidas	El tiempo de ejecución, el número total de registros que entraban al filtro del estado y un array del último registro con la siguiente información (año de recopilación, fecha de carga, tipo de fuente, nombre del departamento, tipo de producto, unidad de medición y valor de medición) . Si no hay registros en ese estado se retorna None.
Implementado (Sí/No)	Si - Grupal

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Saca el tamaño de la lista que tiene los nombres de los estados	$O(1)$
Inicia una nueva lista	$O(1)$
Busca los elementos que tienen el mismo estado que el dado por el usuario con <code>get_element()</code>	$O(1)$ pero como está dentro de un for se convierte en $O(N)$
Añade al final de la lista creada anteriormente el dato que cumple con el filtro	$O(1)$
Usa <code>get_element()</code> para encontrar los elementos de la misma fila del elemento que cumple el filtro	$O(1)$
TOTAL	$O(N)$

Pruebas Realizadas

- Estado buscado: Kansas

Procesador	Apple M1
Memoria RAM	8 GB
Sistema Operativo	macOS Sequoia Versión 15.1

Tablas de datos

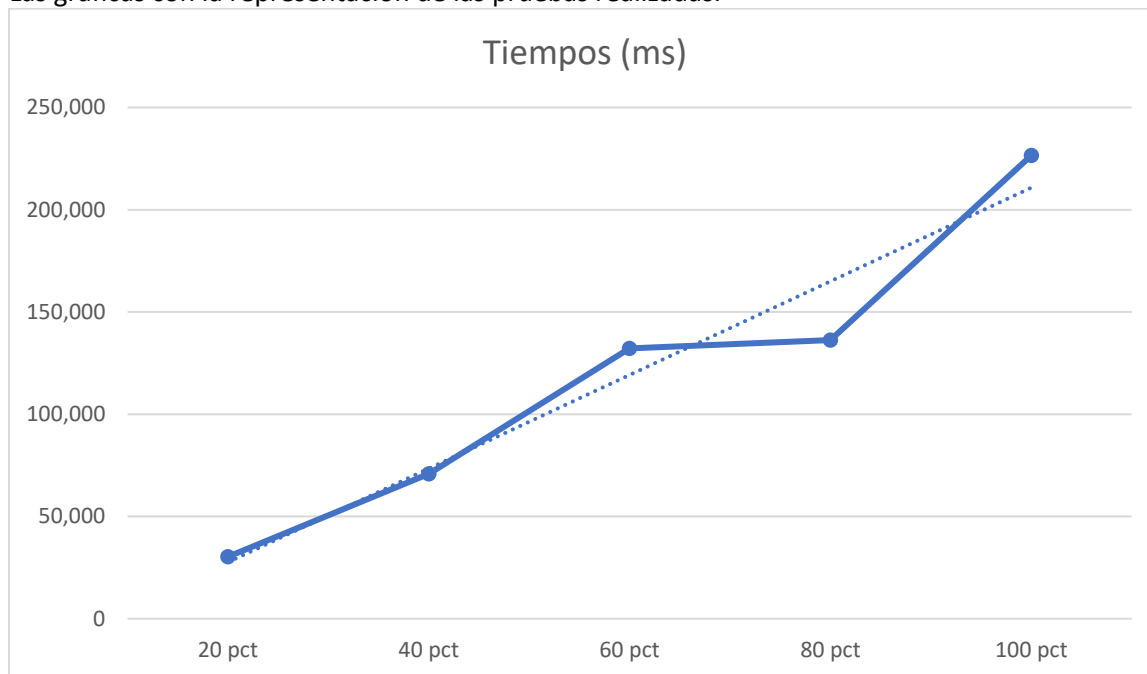
Las tablas con la recopilación de datos de las pruebas.

Entrada	Tiempo (ms)
20%	30.291 ms

40%	70.814 ms
60%	132.21 ms
80%	136.252 ms
100%	226.513 ms

Graficas

Las gráficas con la representación de las pruebas realizadas.



Análisis

Análisis de resultados de la implementación, tener cuenta las pruebas realizadas y el análisis de complejidad.

El algoritmo tiene una complejidad de **O (N)** debido al recorrido que hace en las listas por el filtro del estado. En la gráfica, se evidencia un incremento en los tiempos de ejecución conforme se incrementa el tamaño de la entrada, lo que concuerda con el análisis teórico. La línea de tendencia confirma esta relación, mostrando un crecimiento menos pronunciado que el de una complejidad lineal pura, pero mayor al de una complejidad constante. El comportamiento es consistente en todos los puntos de medición, lo que indica que el rendimiento del algoritmo se mantiene dentro de lo esperado.

Requerimiento <<3>>

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

Breve descripción de como abordaron la implementación del requerimiento

Entrada	El catálogo con la información del CVS, el estado que quiere filtrar el usuario, el año inicial del rango y el año final del rango.
Salidas	El tiempo de ejecución, el número total de registros que entraban al filtro del estado y el rango de años, total de registros tipo "SURVEY", total de registros tipo "CENSUS" y un array del último registro con la siguiente información (tipo de fuente, año de recopilación, fecha de carga, frecuencia de recopilación, tipo de producto y unidad de medición. Si no hay registros en ese estado y en ese rango se retorna None.
Implementado (Sí/No)	Si - Juan Camilo Cancelado

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Se saca el tamaño del arreglo que contiene los estados	O(1)
Se inicia una nueva lista	O(1)
Se saca el elemento de la posición "i" del arreglo de los años de recolección de datos usando get_element() y se guarda en una variable	O(1) pero como está dentro de un for se convierte en O(N)
Se saca el elemento de la posición "i" del arreglo de los estados usando get_element() y se guarda en una variable	O(1) pero como está dentro de un for se convierte en O(N)
Se añade al final de la lista creada el principio los datos que cumplen un filtro	O(1) pero como está dentro de un for se convierte en O(N)
Se añaden a esa lista los datos de la posición "i" que se buscan usando get_element()	O(1) pero como está dentro de un for se convierte en O(N)
Se usa get_element() para encontrar si un dato es survey o census	O(1) pero como está dentro de un for se convierte en O(N)
Se inicia una nueva lista recortada de la anterior	O(1)
Se añade al final de la lista recortada el elemento de la posición "i" de la lista anterior	O(1) pero como está dentro de un for se convierte en O(M) donde M es el número de elementos de la lista de elementos que cumplen en filtro
Añade al final de la lista recortada otros datos necesarios para el retorno	O(1)
TOTAL	O(N+M)

Pruebas Realizadas

- Año inicial: 1997
- Año final: 2022
- Estado : OHIO

Procesador	Apple M1
Memoria RAM	8 GB
Sistema Operativo	Windows 11 Core i5

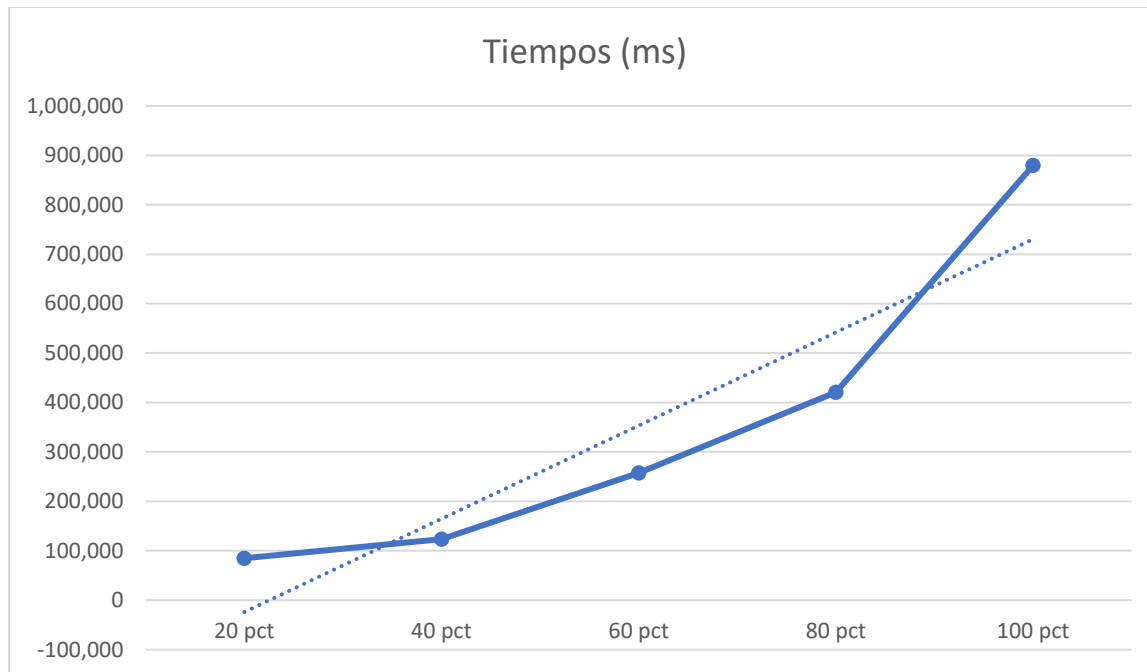
Tablas de datos

Las tablas con la recopilación de datos de las pruebas.

Entrada	Tiempo (ms)
20%	84.9289 ms
40%	134.520ms
60%	257.801ms
80%	420.629ms
100%	879.391ms

Graficas

Las gráficas con la representación de las pruebas realizadas.



Análisis

Análisis de resultados de la implementación, tener cuenta las pruebas realizadas y el análisis de complejidad.

El análisis de complejidad muestra que el algoritmo tiene un orden de **$O(N+M)$** , ya que el algoritmo depende de dos variables de entrada que son los dos filtros como el estado y el rango de años. En la gráfica, se observa que el tiempo de ejecución aumenta de manera casi proporcional al tamaño de los datos procesados. La línea de tendencia sigue este mismo patrón, lo que confirma que el algoritmo presenta un crecimiento lineal, sin embargo observando la inclinación de esta se puede evidenciar si M tiene un impacto más alto en el tiempo de ejecución, que en este caso no es mucho. No se presentan anomalías significativas en la gráfica, lo que indica que la implementación mantiene una ejecución consistente sin variaciones inesperadas.

Requerimiento <<4>>

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

Breve descripción de como abordaron la implementación del requerimiento

Entrada	El catálogo con la información del CVS, el tipo de producto que quiere filtrar el usuario, el año inicial del rango y el año final del rango.
Salidas	El tiempo de ejecución, el número total de registros que entraban al filtro del tipo de producto y el rango de años, total de registros con tipo "SURVEY", total de registros de tipo "CENSUS" y un array para cada registro que cumple el filtro con la siguiente información (tipo de fuente, año de recopilación, fecha de carga, frecuencia de recopilación, nombre del departamento, unidad de medición) Si no hay registros con el tipo de producto en ese rango de años se retorna None.
Implementado (Sí/No)	Si - Gabriela Gomez

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Se saca el tamaño del arreglo que contiene los productos	O(1)
Se inicia una nueva lista	O(1)
Se saca el elemento de la posición "i" del arreglo de los años de recolección de datos usando get_element() y se guarda en una variable	O(1) pero como está dentro de un for se convierte en O(N)
Se saca el elemento de la posición "i" del arreglo de los productos usando get_element() y se guarda en una variable	O(1) pero como está dentro de un for se convierte en O(N)
Se añade al final de la lista creada el principio los datos que cumplen un filtro	O(1) pero como está dentro de un for se convierte en O(N)
Se añaden a esa lista los datos de la posición "i" que se buscan usando get_element()	O(1) pero como está dentro de un for se convierte en O(N)
Se usa get_element() para encontrar si un dato es survey o census	O(1) pero como está dentro de un for se convierte en O(N)
Se inicia una nueva lista recortada de la anterior	O(1)
Se añade al final de la lista recortada el elemento de la posición "i" de la lista anterior	O(1) pero como está dentro de un for se convierte en O(M) donde M es el número de elementos de la lista de elementos que cumplen en filtro
Añade al final de la lista recortada otros datos necesarios para el retorno	O(1)

TOTAL	<i>$O(N)$ M es muy pequeño comparado a N, por lo tanto la complejidad se puede dejar como $O(N)$</i>
--------------	--

Pruebas Realizadas

- Año inicial: 1957
- Año final: 2019
- Tipo de producto : Cattle

Procesador	Apple M1
Memoria RAM	8 GB
Sistema Operativo	Windows 11 Corei5

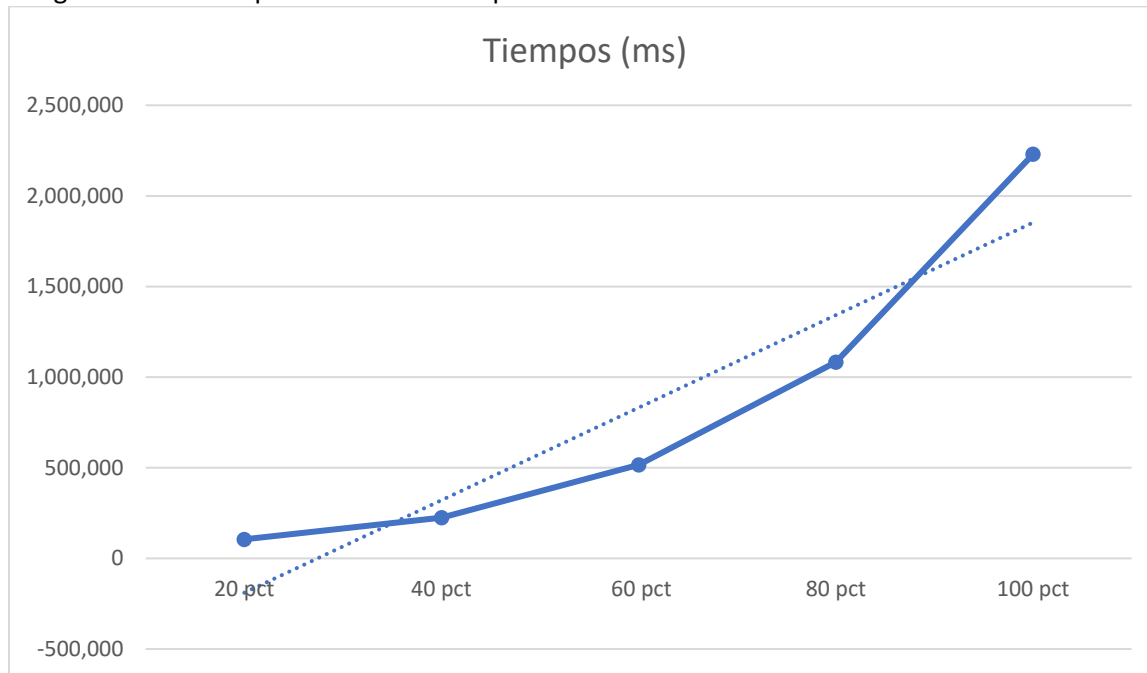
Tablas de datos

Las tablas con la recopilación de datos de las pruebas.

Entrada	Tiempo (ms)
20%	105.397 ms
40%	225.258 ms
60%	516.881 ms
80%	1082.629 ms
100%	2229.743 ms

Graficas

Las gráficas con la representación de las pruebas realizadas.



Análisis

Análisis de resultados de la implementación, tener cuenta las pruebas realizadas y el analisis de complejidad.

El análisis de complejidad indica un comportamiento **O(N)**, aunque esta dependa de dos variables como la anterior sin embargo al M ser muy pequeña no causa ningun cambio significativo. En la gráfica, se observa un crecimiento constante del tiempo de ejecución conforme aumenta el número de datos procesados, lo que confirma la relación lineal predicha en el análisis teórico. La línea de tendencia refuerza esta conclusión al seguir un patrón estable sin fluctuaciones abruptas, demostrando que el tiempo de procesamiento depende directamente del tamaño de la entrada.

Requerimiento <<5>>

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

Breve descripción de como abordaron la implementación del requerimiento

Entrada	El catálogo con la información del CVS, la categoría que quiere filtrar el usuario, el año inicial del rango y el año final del rango.
Salidas	El tiempo de ejecución, el número total de registros que entraban al filtro de la categoría y el rango de años, total de registros con tipo "SURVEY", total de registros de tipo "CENSUS" y un array para cada registro que cumple el filtro con la siguiente información (tipo de fuente, año de recopilación, fecha de carga, frecuencia de recopilación, nombre del departamento, unidad de medición, tipo del producto) Si no hay registros con la categoría en ese rango de años se retorna None.
Implementado (Sí/No)	Si – Pedro Archila

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Iniciar una nuevo array_list	$O(1)$
Encontrar el tamaño de la lista y comprarlo con la posición	$O(1)$ pero como es la condición de un while, es $O(N)$
Agregar un elemento que cumple el filtro al final de la lista creada	$O(1)$, pero como está dentro de un while, es $O(N)$
Revisar el tamaño de la lista de elementos que cumplen el filtro (el peor caso es que el tamaño sea mayor a 20)	$O(1)$
Si el tamaño es mayor a 20, crea una nueva lista vacía	$O(1)$
Añade los primeros 5 y los últimos 5 elementos de la lista grande al final de la nueva lista	$O(1)$ pero como está dentro de un for, es $O(M)$ siendo M la cantidad de elementos en la lista que cumplen los filtros
TOTAL	$O(N)$ M es muy pequeño comparado a N, por lo tanto la complejidad se puede dejar como $O(N)$

Pruebas Realizadas

- Año inicial: 1950
- Año final: 2024
- Categoría Estadística: Inventory

Procesador	Apple M1
Memoria RAM	8 GB
Sistema Operativo	macOS Sequoia Versión 15.1

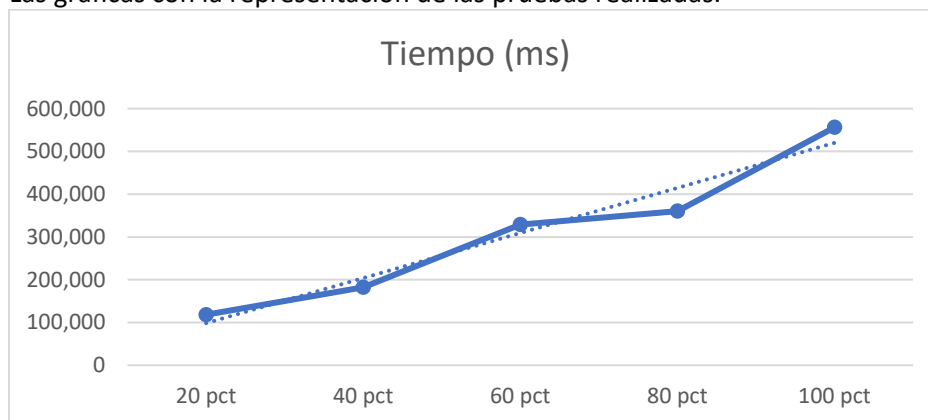
Tablas de datos

Las tablas con la recopilación de datos de las pruebas.

Entrada	Tiempo (ms)
20%	118.149 ms
40%	182.126 ms
60%	329.144 ms
80%	360.533 ms
100%	556.046 ms

Graficas

Las gráficas con la representación de las pruebas realizadas.



Análisis

Análisis de resultados de la implementación, tener cuenta las pruebas realizadas y el análisis de complejidad.

El algoritmo tiene una complejidad $O(N + M)$, pero dado que M es pequeño en comparación con N , su impacto en el tiempo de ejecución es mínimo. En la gráfica, se observa un crecimiento lineal del tiempo con respecto a N , lo que confirma que el desempeño está dominado por la cantidad de registros en la lista principal. La línea de tendencia sigue un patrón estable y proporcional, sin desviaciones notables, lo que indica que el algoritmo mantiene un comportamiento predecible.

Requerimiento <<6>>

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

Breve descripción de como abordaron la implementación del requerimiento

Entrada	El catálogo con la información del CVS, el departamento que quiere filtrar el usuario, el año inicial del rango y el año final del rango.
Salidas	El tiempo de ejecución, el número total de registros que entraban al filtro del departamento y el rango de años, total de registros con tipo "SURVEY", total de registros de tipo "CENSUS" y un array para cada registro que cumple el filtro con la siguiente información (tipo de fuente, año de recopilación, fecha de carga, frecuencia de recopilación, nombre del departamento, unidad de medición, tipo del producto) Si no hay registros con el departamento en ese rango de años se retorna None.
Implementado (Sí/No)	Si – Grupal

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Iniciar una nueva array_list	O(1)
Encontrar el tamaño de la lista y comprarlo con la posición	O(1) pero como es la condición de un while, es O(N)
Agregar un elemento que cumple el filtro al final de la lista creada	O(1), pero como está dentro de un while, es O(N)
Revisar el tamaño de la lista de elementos que cumplen el filtro (el peor caso es que el tamaño sea mayor a 20)	O(1)
Si el tamaño es mayor a 20, crea una nueva lista vacía	O(1)
Añade los primeros 5 y los últimos 5 elementos de la lista grande al final de la nueva lista	O(1) pero como está dentro de un for, es O(M) siendo M la cantidad de elementos que cumplen los filtros
TOTAL	<i>O(N) M es muy pequeño comparado a N, por lo tanto la complejidad se puede dejar como O(N)</i>

Pruebas Realizadas

Las pruebas realizadas fueron realizadas con la siguiente información:

- Fecha inicial: 1950-01-01
- Fecha final: 2024-12-12
- Estado: California

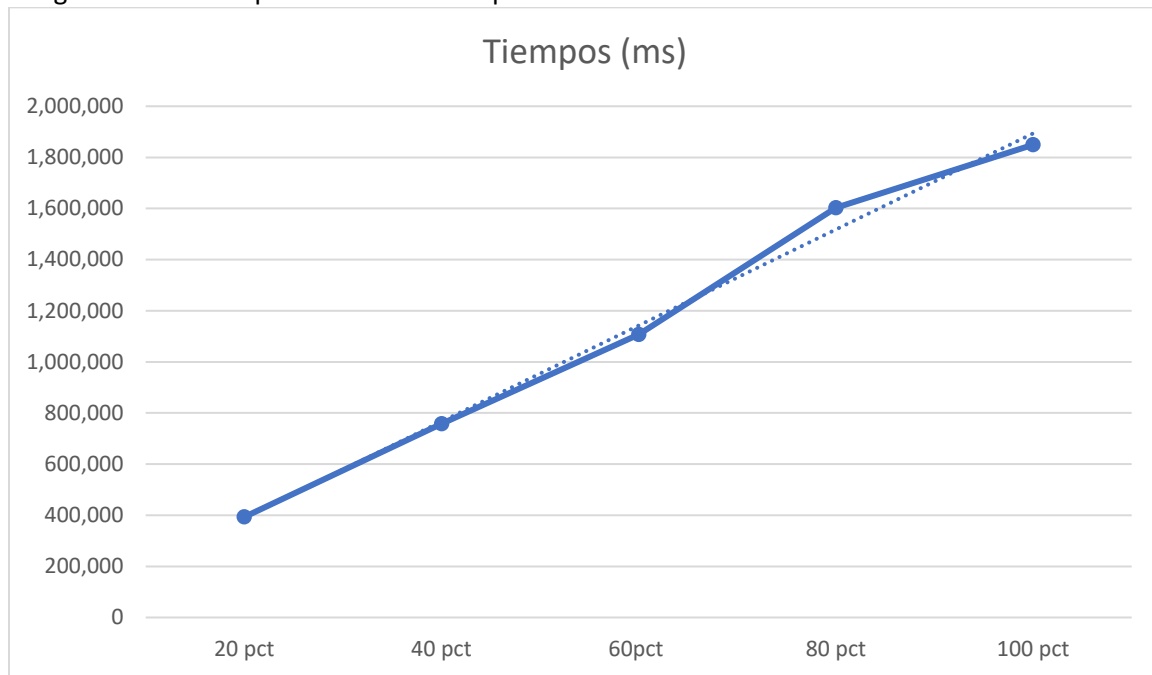
Procesador	Apple M1
Memoria RAM	8 GB
Sistema Operativo	macOS Sequoia Versión 15.1

Tablas de datos

Entrada	Tiempo (ms)
20%	393.105 ms
40%	757.685 ms
60%	1107.783 ms
80%	1601.934 ms
100%	1849.295 ms

Graficas

Las gráficas con la representación de las pruebas realizadas.



Análisis

Análisis de resultados de la implementación, tener cuenta las pruebas realizadas y el analisis de complejidad.

El algoritmo tiene una complejidad **$O(N)$** , ya que recorre la lista de registros para agruparlos según un criterio específico. La gráfica muestra un crecimiento progresivo de los tiempos de ejecución a medida que aumenta el tamaño del conjunto de datos. La línea de tendencia sigue una trayectoria lineal, validando que el procesamiento de los registros escala de manera proporcional con la cantidad de datos evaluados. No se observan irregularidades en los tiempos medidos, lo que indica que la ejecución del algoritmo es estable y coherente con su análisis de complejidad, sin dejar de lado el hecho de que no es $O(N+M)$ ya que esta M es muy pequeña por lo que el algoritmo depende mas de los registros.

Requerimiento <<7>>

Plantilla para el documentar y analizar cada uno de los requerimientos

Descripción

Breve descripción de como abordaron la implementación del requerimiento

Entrada	El catálogo con la información del CVS, el departamento que quiere filtrar el usuario, el año inicial del rango y el año final del rango.
Salidas	El tiempo de ejecución, el número total de registros que entraban al filtro del departamento y el rango de años, y un array para los registros con mayor y menor ingresos con la siguiente información (año de recopilación, indicación si es "MAYOR", "MENOR", valor de ingresos, total de registros que cumplen filtro, total de registros que cumplen \$ pero no son válidos, total de registros "SURVEY" y total de registros "CENSUS"). Si no hay registros ni mayores ni menores con el departamento en ese rango de años se retorna None.
Implementado (Sí/No)	Si – Grupal

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Inicialización de variables	$O(1)$
Ciclo externo sobre años	$O(A)$ (A siendo la diferencia de años)
Ciclo interno sobre Registros	$O(N)$ (N siendo tamaño de la columna "states")
Acceder a variables, asignaciones, incrementos, comparaciones y uso de funciones nativas de python para la recolección de datos	$O(1)$ para toda variable del ciclo interno
Agregar elementos a la lista	$O(A)$ en el bucle externo
Búsqueda de mayor y menor valor	$O(A)$
TOTAL	$O(A \times N)$, A en la mayoría de los casos en este ejercicio es significativamente menor a N (A puede ser máximo 2025, y N puede llegar a

	<i>ser 10 millones), por lo cual se podría aproximar a $O(N)$.</i>
--	---

Pruebas Realizadas

- Año inicial: 1950
- Año final: 2024
- Estado buscado: Florida

Procesador	Apple M1
Memoria RAM	8 GB
Sistema Operativo	macOS Sequoia Versión 15.1

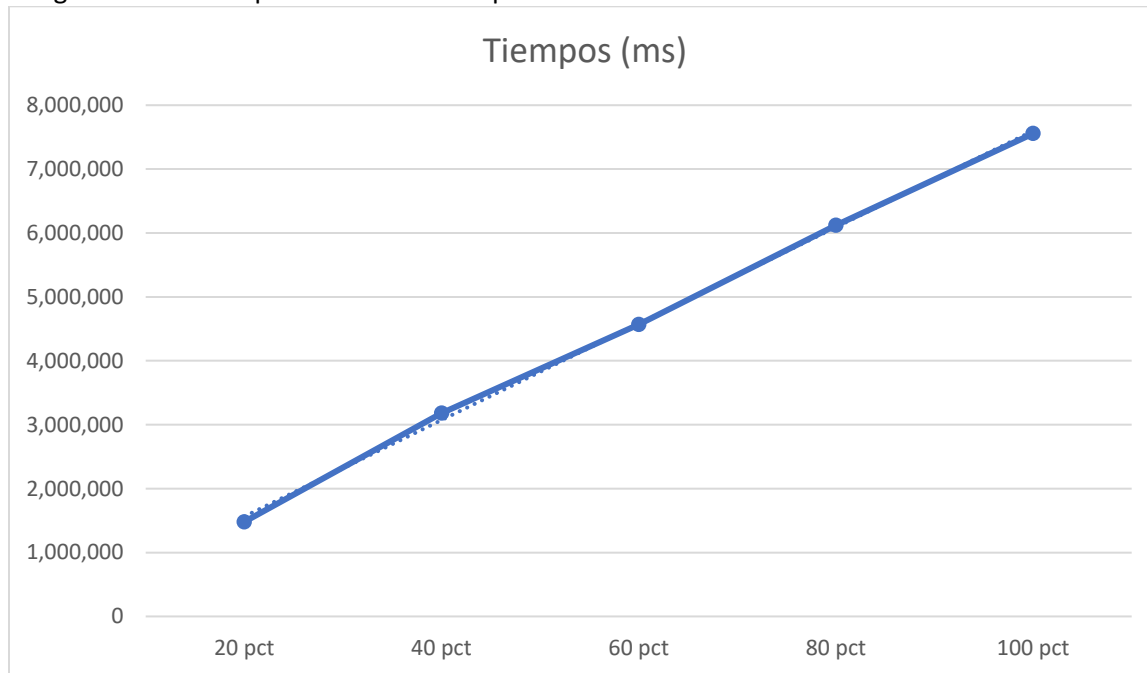
Tablas de datos

Las tablas con la recopilación de datos de las pruebas.

Entrada	Tiempo (ms)
20%	1479.956 ms
40%	3183.452 ms
60%	4566.26 ms
80%	6120.27 ms
100%	7555.615 ms

Graficas

Las gráficas con la representación de las pruebas realizadas.



Análisis

Análisis de resultados de la implementación, tener cuenta las pruebas realizadas y el analisis de complejidad.

El algoritmo tiene una complejidad $O(A \times N)$, donde A representa la cantidad de años en el rango de consulta y N el número total de registros. Dado que A es significativamente menor que N (con un máximo de 2025, mientras que N puede llegar a 10 millones), el comportamiento del algoritmo se asemeja a $O(N)$. En la gráfica, se observa un crecimiento lineal en el tiempo de ejecución, lo que confirma que el rendimiento está determinado principalmente por N . La línea de tendencia sigue este patrón sin variaciones significativas, lo que indica que el impacto de A es constante y no altera la relación directa entre el tamaño de los datos y el tiempo de procesamiento. Esto valida que, en la práctica, el algoritmo se comporta como $O(N)$.